Software Architecture and Design Specification

Project: Online Bookstore

Version: 1.0

Authors: Group 2

Date: 05-10-2026

Status: Draft for Submission

A multi-seller e-commerce platform for buying and selling new and second-hand books.

PES University, Bengaluru | UE24CS341A - Software Engineering | Semester V

Revision History

Version	Date	Author	Change Summary
0.1	03 Sep 2026	Group 2	Initial SAD draft aligned with the assigned Online Bookstore project.
0.5	05 Oct 2026	Group 2	Architecture, component design, security architecture, API design and sequence flows prepared from SRS v1.0.
1.0	05 Oct 2026	Group 2	SAD baseline prepared for Mini-Project Deliverables Part-1.

Approvals

Role	Name	Signature / Date
Project Team	Group 2	_____ / 05-10-2026
Faculty / Reviewer	_____	_____

Document Overview

This document describes the software architecture and detailed design of the Online Bookstore. It translates the approved SRS into architectural components, interfaces, security controls, design flows and traceability. The architecture is based on a React client, Node.js/Express service layer and MongoDB data store, with Redis, a payment gateway, SMTP e-mail and cloud object storage as supporting interfaces.

Contents

1. Introduction

2. Document Overview

3. Architecture

4. Design

5. Appendices

1. Introduction

1.1 Purpose

This Software Architecture and Design Specification (SAD) defines the architecture and design of the Online Bookstore described in the approved SRS. It identifies the major software components, their responsibilities, interactions, technology choices, security controls, design interfaces and representative execution flows.

1.2 Scope

The SAD covers version 1.0 of the Online Bookstore: account and access management; catalogue, search and reviews; cart, wishlist, checkout, payment and order tracking; seller listing and fulfilment operations; administrator moderation and reporting; and the supporting persistence, caching, notification and external-service interfaces.

1.3 Audience

Developers implementing the React, Node.js/Express and MongoDB solution.

QA/test engineers deriving integration and system tests from the design.

Project faculty and reviewers evaluating architecture and requirement coverage.

Security reviewers checking authentication, authorization, payment and audit controls.

Maintainers who need to understand component responsibilities and dependencies.

1.4 Definitions

Term	Meaning
MERN	MongoDB, Express.js, React and Node.js technology stack.
JWT	JSON Web Token used for authenticated access.
RBAC	Role-Based Access Control.
OTP	Six-digit One-Time Password used for e-mail verification.
Listing	One seller's offer of one book title, with its own condition, price and stock.
ISBN-13	Thirteen-digit identifier used to identify a catalogue title.
NFR	Non-functional requirement.
RTM	Requirement Traceability Matrix.

1.5 References

Software Requirements Specification - Online Bookstore, Group 2, Version 1.0 approved, 10 September 2026.

IEEE Std 830-1998, Recommended Practice for Software Requirements Specifications.

Project Guidelines for UE24CS341A, Semester V 2026.

MongoDB documentation for text indexes and aggregation pipeline.

OWASP Top 10 (2021).

2. Document Overview

2.1 How to Use This Document

Section 3 should be used to understand the system architecture, component responsibilities, architecture pattern, technology stack, risks, requirement traceability and security architecture. Section 4 should be used for implementation-level design, representative API contracts, sequence flows, error handling and user-experience decisions. The appendices provide traceability, glossary material and design assumptions.

2.2 Related Documents

Online Bookstore SRS Version 1.0.

Requirement Traceability Matrix in Appendix C of the SRS.

Test Plan / Software Test Plan for the Online Bookstore.

Appendix B field layouts and report requirements workbook referenced by the SRS.

3. Architecture

3.1 Goals & Constraints

The architecture is designed to satisfy the approved requirements without unnecessary complexity. Primary goals are secure authentication and authorization, reliable order and stock processing, responsive catalogue search, maintainable modular services, testability through injectable interfaces, and reuse through a shared React component library.

Category	Architecture goal / constraint
Performance	Catalogue search within 2 seconds at the 95th percentile under 10,000 listings and 200 concurrent users; catalogue/detail pages within 3 seconds.
Availability	At least 99% availability during the evaluation window, measured through a health-check monitor.
Security	TLS 1.2+, bcrypt password hashes, short-lived access tokens, RBAC, server-side validation, no card-data storage and append-only audit logging.
Data integrity	Atomic stock decrement at payment confirmation; no hard deletion of order/payment/shipment history.
Technology	MERN stack is fixed by the project brief; MongoDB Atlas and Redis are used as specified by the SRS.
Delivery	Four part-time developers must implement, test and demonstrate the product within one semester.
Operations	Third-party services remain within free tiers; source is managed in one protected GitHub repository with CI gates.

3.2 Stakeholders & Concerns

Stakeholder	Primary concerns
Buyer	Fast discovery, correct pricing/stock, secure checkout, order visibility and usable responsive UI.
Seller	Simple listing/inventory management, fulfilment workflow, low-stock alerts and accurate settlement reports.
Administrator	Moderation, auditability, user management, platform analytics and exports.
Developers	Clear module boundaries, testability, maintainability and CI quality gates.
Faculty / Evaluators	Traceability to SRS requirements, demonstrable functionality and measurable quality attributes.
Security / Compliance reviewers	TLS, password protection, RBAC, payment isolation, audit logs and OWASP-aligned validation.

3.3 Component (UML) Diagram

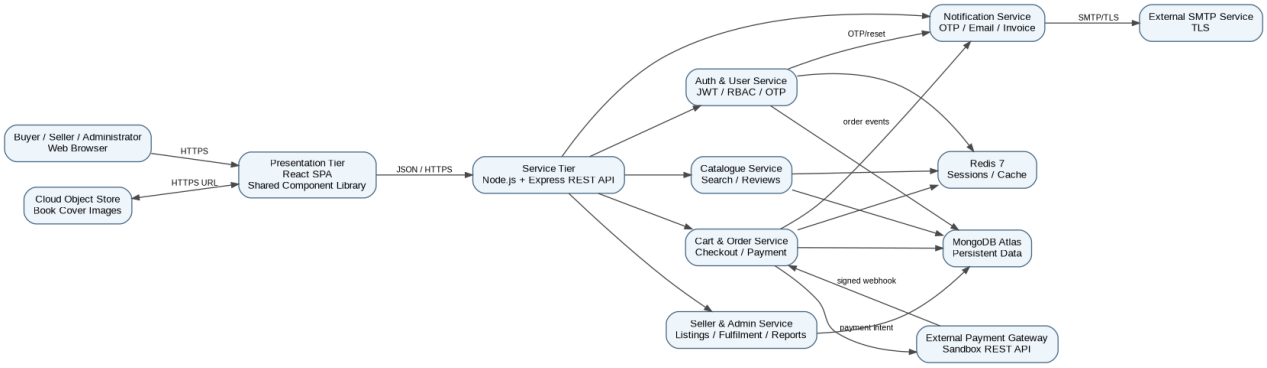


Figure 1. Component architecture - Online Bookstore

The component view implements the three-tier perspective stated in the SRS. The React presentation tier communicates with the Node.js/Express REST service tier over HTTPS/JSON. The service tier is split by responsibility into authentication/user, catalogue/search/review, cart/order/payment, seller/administrator and notification services. MongoDB is the primary persistent store; Redis provides session storage and query caching; payment and SMTP remain outside the application trust boundary.

3.4 Component Descriptions

Component	Responsibilities	Main requirements supported
React Presentation Tier	Buyer, seller and administrator screens; shared components; responsive layout; form validation feedback; authenticated navigation.	FR-01 to FR-24; NFR-04, NFR-07
Auth & User Service	Registration, OTP verification, login/logout, JWT/refresh tokens, password reset, profile/address management and RBAC.	FR-01 to FR-06; NFR-03, NFR-08
Catalogue Service	Catalogue paging, text search, filtering/sorting, book detail, reviews and related-title selection.	FR-07 to FR-12; NFR-01
Cart & Order Service	Cart/wishlist, checkout validation, order lifecycle, payment coordination, invoice triggering and cancellation/return.	FR-13 to FR-18; NFR-05
Seller & Admin Service	Listing/inventory management, low-stock handling, fulfilment, seller dashboard, moderation and platform reports.	FR-19 to FR-24; NFR-02, NFR-06
Notification Service	OTP, order confirmation, invoice and low-stock e-mail delivery; queue/retry handling.	FR-02, FR-04, FR-17, FR-20, FR-21
MongoDB Atlas	Persistent collections for users, addresses, books, listings, carts, orders, payments, shipments, reviews and notifications.	Persistent system data
Redis	Session storage and query caching for repeated filter/search combinations.	NFR-01 and session support
Payment Gateway	Sandbox authorization/capture; application receives payment status through a signed callback.	FR-16; payment security
SMTP Service	TLS e-mail delivery for OTPs, confirmations, invoices and alerts.	FR-02, FR-04, FR-17, FR-20, FR-21
Cloud Object Store	Storage/retrieval of book cover images.	External interface 3.2

3.5 Chosen Architecture Pattern and Rationale

The system uses a three-tier layered architecture with modular services inside the application/service tier. This is consistent with the SRS product perspective: React SPA as the client, Node.js/Express as the REST service layer, and MongoDB as the data store, with Redis and external services supporting the application. The pattern

separates presentation, business/service logic and persistence concerns, while the internal service boundaries keep authentication, catalogue, ordering and seller administration independently testable.

Presentation layer: React SPA and shared component library.

Service layer: Express REST API and responsibility-oriented service modules.

Persistence/integration layer: MongoDB, Redis, payment gateway, SMTP and object storage.

External systems are isolated behind interfaces so payment and e-mail can be stubbed in tests.

A microservices deployment is not required for the current semester-scale system. Keeping the modules within one service deployment reduces operational overhead while preserving clear logical boundaries and future extraction points.

3.6 Technology Stack & Data Stores

Layer / Area	Technology	Purpose
Client	React	Single-page responsive application and shared UI components.
Server	Node.js 20 LTS + Express.js	REST API and service orchestration.
Database	MongoDB 7 / MongoDB Atlas	Primary persistent document store.
ODM	Mongoose 8	Application-side document modelling and database access.
Cache / Session	Redis 7	Session storage and five-minute query/filter cache.
Authentication	JWT + bcrypt	Access/refresh token model and password hashing.
Payment	Sandbox REST payment gateway	Payment intent, authorization/capture and callback.
Notification	SMTP over TLS	OTP, order confirmation, invoice and alert e-mail.
Images	Cloud object store	Book cover image storage.
API contract	OpenAPI document	Authoritative REST API contract maintained in repository.
CI / Quality	ESLint + unit tests	Zero ESLint errors and minimum 70% service-module unit-test coverage.

3.7 Risks & Mitigations

Risk	Impact	Mitigation
Payment gateway unavailable	Checkout cannot complete.	Leave order in PLACED state, do not decrement stock, display retry path and rely on sandbox availability assumption.
SMTP unavailable	OTP/notification delivery delayed.	Queue notification and retry up to three times with exponential backoff; expose resend for OTP.
Redis unavailable	Performance degradation and session/cache disruption.	Use read-through caching and database fallback for session storage as stated in SRS.
Concurrent purchase of last stock	Overselling and data inconsistency.	Atomic stock decrement at payment confirmation; verify with concurrent system test.
Unauthorized route/object access	Privacy or privilege violation.	Server-side RBAC and object-level ownership checks on every protected read.
Database/service failure	Availability loss.	Managed database, health checks, daily automated backup and restore rehearsal before demonstration.
Third-party free-tier limits	Resource exhaustion.	Monitor storage/e-mail/container usage and keep external integrations within free-tier constraints.

3.8 Traceability to Requirements

The architecture mapping below summarizes the SRS traceability approach. The complete SRS RTM maps FR-01 to FR-24 and NFR-01 to NFR-08 to architecture, design, code and test artefacts.

Requirement	Architecture reference	Design responsibility
FR-01–FR-06	Auth & User Service	Registration, OTP, JWT, profile/address and RBAC
FR-07–FR-12	Catalogue Service	Paging, search, filters, detail, reviews and related titles
FR-13–FR-18	Cart & Order Service	Cart, wishlist, checkout, payment, invoice and order lifecycle
FR-19–FR-24	Seller & Admin Service	Listings, inventory, fulfilment, settlement, moderation and analytics
NFR-01	Catalogue + Redis	Indexed search and cache under reference load
NFR-02	Deployment + monitoring	Health endpoint and uptime monitoring
NFR-03	Auth + API + payment integration	TLS, bcrypt, token expiry and payment isolation
NFR-04	React Presentation Tier	Four-screen checkout, 360px usability and WCAG AA
NFR-05	Cart & Order Service + MongoDB	Atomic stock decrement
NFR-06	All service modules + CI	ESLint and unit-test coverage gate
NFR-07	React Presentation Tier	Cross-browser compatibility
NFR-08	Auth/Order/Seller/Admin + Audit Log	Append-only administrative/payment audit events

3.9 Security Architecture

Security controls are layered across the client, service tier, data tier and external integrations. The payment gateway remains outside the application boundary so that card data does not enter or get stored by the bookstore.

Threat / Concern	Control	Design location
Spoofing / credential theft	bcrypt cost 12, e-mail OTP verification, short-lived access tokens and refresh-token rotation.	Auth & User Service
Broken authorization / privilege escalation	Server-side RBAC on every protected route and object-level ownership checks.	Express middleware / services
Tampering	HTTPS/TLS, server-side validation/sanitisation, signed payment webhook verification and idempotent callback handling.	API/service layer
Information disclosure	No card data stored; sensitive fields excluded from logs; HTTPS; controlled object ownership.	API, payment integration, data access
Injection / XSS	Server-side validation and sanitisation; OWASP Top 10 review.	All API inputs
Denial of Service / abuse	Failed-login rate limiting and OTP request cap; health monitoring.	Auth service / deployment
Repudiation	Append-only audit log for every administrative action and payment event.	Audit logging
Data loss	Daily automated backups retained for seven days and restore rehearsal.	MongoDB Atlas / operations

4. Design

4.1 Design Overview

The detailed design follows the architectural boundaries in Section 3. Each service exposes focused operations through the Express REST layer. Persistent data is stored in the eleven collections defined by the SRS. Service-layer validation is used for relationships because MongoDB does not enforce relational foreign keys. Order lines store the unit price at purchase time so later seller price changes cannot modify historical order values.

4.1.1 Key Data and Service Responsibilities

Design area	Decision
Identity	USER is a single collection with a role attribute; a person may be both buyer and seller, while administrator is separate.
Catalogue	BOOK identifies a title by ISBN-13; LISTING connects a title to a seller and carries condition, price and stock.
Orders	ORDER stores lifecycle state; ORDER_ITEM freezes unit price at purchase time and references the listing.
Payment	PAYMENT records attempts against an order; card details remain exclusively with the payment gateway.
Notification	NOTIFICATION records outbound messages and delivery outcome for audit/retry handling.
Caching	Redis caches repeated filter combinations with a five-minute TTL and supports session storage.
Reporting	Aggregation queries generate seller and administrator reports; exports are CSV/PDF as specified.

4.2 UML Sequence Diagrams

Two representative sequences are included below. They cover account activation and the critical checkout/payment flow specified in the SRS.

4.2.1 Sequence Diagram - Registration and OTP Verification

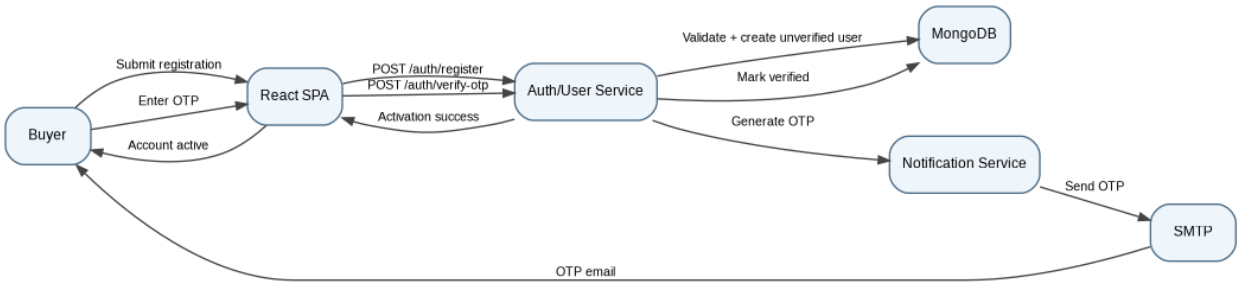


Figure 2. Registration, OTP generation, e-mail delivery and verification

The registration flow creates an unverified user, generates a six-digit OTP and sends it through the notification/SMTP path. Verification activates the account. The SRS further constrains OTP validity to ten minutes, limits resend frequency, and blocks unverified accounts from signing in.

4.2.2 Sequence Diagram - Checkout and Payment Confirmation

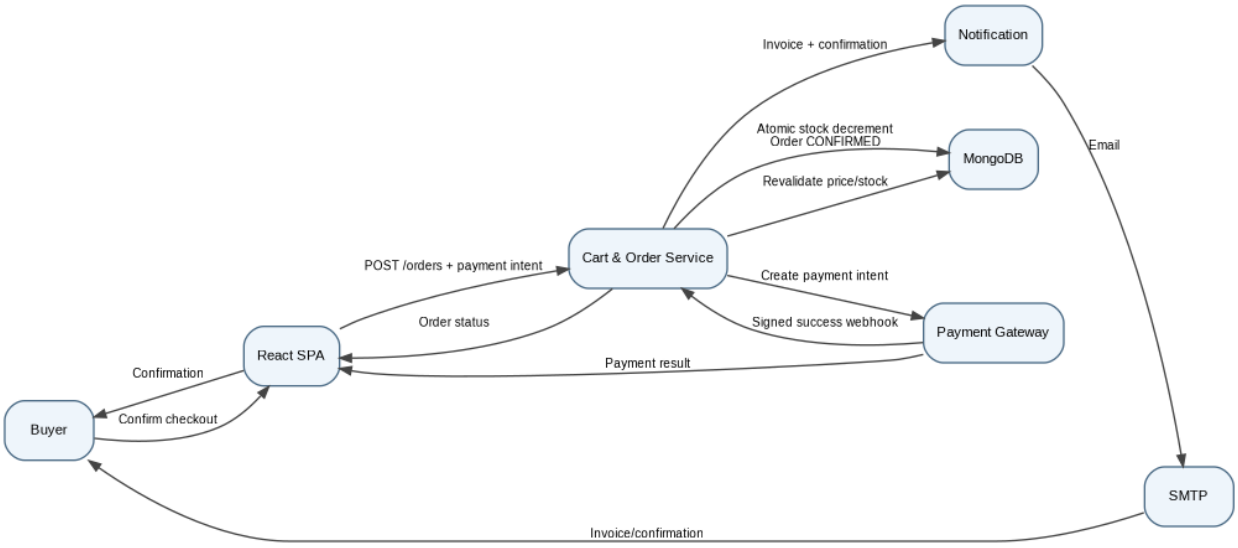


Figure 3. Checkout, payment gateway callback, stock update and notification

The checkout flow revalidates current price and stock before creating a payment intent. The application trusts only a signature-verified gateway callback. On success, stock is decremented atomically and the order moves to CONFIRMED; on failure the order remains PLACED and stock is not decremented. The callback is idempotent so repeated callbacks do not confirm an order twice.

4.3 API Design

The SRS specifies a REST API over HTTPS with JSON requests/responses and bearer JWT authentication. The following are representative design interfaces derived from the SRS requirements; the repository OpenAPI document remains the authoritative implementation contract.

Component / Endpoint	Method	Request / Inputs	Response / Outputs	Key errors / controls
Auth /api/auth/register	POST	name, email, phone, password, role	Unverified user; OTP dispatch status	400 validation/duplicate email
Auth /api/auth/verify-otp	POST	email, otp	Verification success/session	400 invalid/expired OTP; 429 request limit
Catalogue /api/books/search	GET	q, category, price, condition, language, rating, sort, page	Paged results	400 invalid filter; indexed search
Cart /api/cart/items	POST	listingId, quantity	Updated cart line/totals	400 invalid quantity/stock; ownership
Order /api/orders	POST	cart, addressId, couponCode	PLACED order + payment intent ref	400 revalidation failure
Payment /api/payments/webhook	POST	Signed gateway callback	Idempotent acknowledgement	401 invalid signature; duplicates ignored
Seller /api/seller/listings	POST	isbn13, condition, price, mrp, stock	Created/updated listing	400 price > MRP; RBAC
Admin /api/admin/reports	GET	month, report type, export format	Report data or CSV/PDF	403 non-admin; aggregation handling

4.3.1 API Design Rules

All application API traffic uses HTTPS with TLS 1.2 or higher; plain HTTP is redirected to HTTPS.

Authenticated requests carry a bearer JWT in the Authorization header.

Errors return an HTTP status, stable machine-readable error code and human-readable message.

The payment webhook verifies its signature before trusting the payload and is idempotent.

CORS accepts only the deployed client origin.

Sensitive information such as passwords, tokens and card details is never written to application logs.

4.4 Error Handling, Logging & Monitoring

Area	Design
Validation	Validate all user input server-side; return field-level validation messages in the client.
Authentication	Return 401 for missing/invalid authentication and 403 for authenticated users without required role/ownership.
Payment	Gateway failure leaves order PLACED, preserves stock and permits the defined retry count.
Notification	Queue failed e-mail and retry up to three times; final failure is recorded.
Database	Use compensating records rather than hard deletion for order/payment/shipment history.
Logging	Do not log passwords, card data or other sensitive authentication material.
Audit	Record every administrative action and payment event with actor, action, target and timestamp in an append-only audit log.
Monitoring	Monitor health/uptime, search latency, payment failure rate, notification failures and critical service errors.

4.5 UX Design

The React client uses a shared component library for buyer, seller and administrator surfaces. The persistent header provides search, cart and account access on buyer-facing screens. The UI is responsive down to a 360 px viewport, uses inline field-level errors, requires confirmation for destructive actions, supports keyboard navigation and targets WCAG 2.1 AA contrast. Checkout is constrained to at most four screens from cart to confirmation.

Buyer surfaces: Home/Catalogue, Search Results, Book Detail, Cart, Checkout, Order Confirmation, Order History/Tracking.

Seller surfaces: Listing Form, Inventory, Seller Dashboard and fulfilment views.

Administrator surfaces: Moderation Queue and Analytics/Reports.

Failed payments explain the failure and expose a retry path instead of showing only a status code.

4.6 Open Issues & Next Steps

Complete and freeze the repository OpenAPI contract for all production routes.

Implement component interfaces with injected payment and SMTP adapters so external services can be stubbed in tests.

Generate final UML diagrams in the team's chosen modelling tool if editable diagram source files are required.

Complete the security review against OWASP Top 10 before demonstration.

Run the 200-concurrent-user search test and verify the 2-second 95th-percentile target.

Run the concurrent last-copy purchase test to demonstrate atomic stock handling.

Complete cross-browser regression and accessibility walkthrough.

Rehearse the database restore once before the final demonstration.

5. Appendices

5.1 Glossary

Term	Meaning
BOOK	Catalogue title identified independently of a seller.
LISTING	Seller-specific offer of a title, carrying condition, price and stock.
ORDER	Purchase record containing the order summary and lifecycle state.
ORDER_ITEM	Order line that freezes the unit price at purchase time.
RBAC	Role-Based Access Control used to restrict protected routes.
JWT	JSON Web Token used for authenticated access.
OTP	Six-digit one-time password used for e-mail verification.
NFR	Non-functional requirement such as performance, security or usability.
RTM	Requirement Traceability Matrix connecting requirements to architecture, code and tests.

5.2 Requirement-to-Design Traceability Extract

Requirement range	Primary design reference	System test coverage
FR-01–FR-06	Auth & User Service	STC-01 to STC-03
FR-07–FR-12	Catalogue Service	STC-04 to STC-06
FR-13–FR-18	Cart & Order Service	STC-07 to STC-09
FR-19–FR-24	Seller & Admin Service	STC-10 to STC-12
NFR-01–NFR-02	Catalogue/Redis + monitoring	STC-13
NFR-03–NFR-04	Security + React UX	STC-14
NFR-05–NFR-06	Order persistence + CI	STC-15
NFR-07–NFR-08	React compatibility + Audit Log	STC-16

The SRS traceability appendix defines sixteen system test cases (STC-01 to STC-16) covering the functional and non-functional requirements. The complete RTM remains the authoritative mapping for code-file and individual test-case references.

5.3 Architecture Decisions

Decision	Choice	Rationale
AD-01	Three-tier layered architecture	Matches the SRS product perspective and separates presentation, service and persistence responsibilities.
AD-02	Modular service boundaries inside Express	Keeps the semester-scale deployment simple while isolating authentication, catalogue, order and seller/admin concerns.
AD-03	MongoDB Atlas as primary store	Matches the fixed MERN stack and the SRS document-oriented collection design.
AD-04	Redis for sessions/cache	Supports NFR-01 performance and the SRS session/cache design.
AD-05	External payment gateway	Prevents card data from entering the application and keeps payment handling outside the application trust boundary.
AD-06	SMTP adapter with retry queue	Provides OTP and transaction notifications while handling temporary delivery failures.
AD-07	Shared React component library	Supports reusability and consistent buyer/seller/admin UI surfaces.

5.4 Tools

Microsoft Word / DOCX for document preparation.

draw.io or PlantUML for editable UML artefacts, if required by the course.

OpenAPI/Swagger for REST contract documentation.

GitHub and CI for version control and quality gates.

Appendix: Project Roles and Responsibilities

The following team assignments apply across the Online Bookstore project documents. Requirement ownership in the SRS remains unchanged. The Product Owner / Faculty Reviewer role remains with the course team/faculty.

Role	Assigned member	Responsibilities
QA / Test Lead	Deeptha S	Coordinate test planning and execution, review coverage, consolidate results and track defects.
Developer / Development Lead	Hardhick M Gowda	Coordinate implementation, support the test environment, analyze defects and provide unit-test evidence.
Test Engineer	Chenchu Sreekrishna Koushik	Design and execute test cases, record evidence and log defects.
Requirements & Documentation Coordinator	G H Pramod	Maintain requirements/document consistency, track requirement changes and coordinate project documentation.

This appendix assigns project coordination roles; it does not change SRS requirement owners.